SCHOOL OF TECHNOLOGY	WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

Design and Implementation of a Database Management System for Research Project, Publication and Grant Management System

Program	B.Tech – Computer Science and Engineering (AI & ML)
Section	AIML Rhinos
Faculty	Sahithi mam
Date of Submission	14-10-2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102087	G. Varun Kumar	7995062452	Varunkumar1435678@gmail.com

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, sahithi mam, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled **"Design and Implementation of a Database Management System for Research Project, Publication and Grant Management System"**, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature

Date	**:**	14-10-2026	**Name:**	G. Varun Kumar
Place	**:**	Woxsen university	**Roll Number:**	25WU0102087
			Program:	B.Tech CSE (AI & ML)

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	12
1.7 Entity and Relationship Summary	13
2. LOGICAL DATABASE DESIGN	14
2.1 Relational Schema	14
2.2 Keys and Integrity Constraints	14
2.3 Functional Dependencies	16
2.4 Normalization up to Third Normal Form (3NF)	17
2.5 Data Dictionary	17
2.6 Implementation of Logical Design: DDL and Sample Data	20
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	22
3.1 Application Overview	22
3.2 Development Environment and Database Connectivity	22
3.3 Application Modules and Database Mapping	23
3.4 CRUD Operations	23
3.5 Business Rules, Validation and Error Handling	24
3.6 Reports and Database Operations	24
3.7 Application Screens	25
3.8 Testing and Results	25
4. CONCLUSION	27
4.1 Limitations and Future Enhancements	27
5. REFERENCES	28
6. APPENDICES	29

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	10
Table 4	Entity and Relationship Summary	13
Table 5	Relational Schema	14
Table 6	Keys and Integrity Constraints	15
Table 7	Normalization Status up to 3NF	17
Table 8	Data Dictionary	18
Table 9	Database Implementation Environment	20
Table 10	Application Development Environment	22
Table 11	Application Module-to-Database Mapping	23
Table 12	CRUD Operations	24
Table 13	Business Rules, Validation and Error Handling	24
Table 14	Reports and Database Operations	25
Table 15	Test Cases	25

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	Project Registration and Investigator Assignment Screen	25
Figure 3	Expenditure Entry and Budget-Rule Rejection	25
Figure 4	Milestone Tracking and Publication Linkage Screen	25
Figure 5	Budget Utilization Report	25

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
BCNF	Boyce-Codd Normal Form
Co-PI	Co-Principal Investigator
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DOI	Digital Object Identifier
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FK	Foreign Key
FR	Functional Requirement
GUI	Graphical User Interface
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
PBL	Project-Based Learning
PI	Principal Investigator
PK	Primary Key
SQL	Structured Query Language

ABSTRACT

Research offices in universities manage projects, investigators, funding agencies, grants, milestones, expenditure, publications and patents, but these records are usually scattered across separate spreadsheets and documents. As a result, project progress, fund utilization and research output are difficult to monitor and verify. This project aims to design and implement a relational database system, with a small Python application, that brings these records together and enforces the rules of research administration. An entity-relationship model with eleven principal entities was converted into thirteen relational tables and normalized to Third Normal Form. The design was implemented in MySQL using primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints, together with triggers that keep expenditure within budget and milestone dates within project dates. The application supports project registration, investigator assignment, grant and budget entry, milestone tracking, expenditure, publication linkage and reporting, and six report views summarize progress, budget utilization, milestones, researcher output, agency portfolio and publications. Testing with valid and invalid inputs showed that correct data is accepted and rule violations are rejected with clear messages. The system gives a research office one reliable source for monitoring research output and fund use.

KEYWORDS

Keywords: Relational Database, ER Diagram, Normalization, MySQL, Python, Research Grant Management

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Research offices in universities must keep track of many connected records: research projects and their investigators, the departments that host them, the funding agencies that sanction grants, the milestones that mark progress, the money spent against each grant, and the publications and patents that result. In many institutions these records are kept in separate spreadsheets and documents. The same project title or investigator name is typed in several files, and nobody can easily check that the files agree with each other.

This manual approach makes it hard for the research office, investigators, finance staff and department heads to answer basic questions: how far each project has progressed, how much of each grant has been spent, which milestones are overdue, and what each researcher or agency has produced. It also allows invalid data, such as a project that ends before it starts, spending above the sanctioned amount, or the same publication recorded twice. A relational database with enforced integrity rules, and a simple application on top of it, is needed to hold all these records in one place and to produce reliable reports.

1.2 Objectives

- Design an ER model covering the 11 principal entities and convert it into a relational schema of 13 tables normalized to 3NF.
- Implement the database in MySQL with PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT constraints and triggers that enforce the five business rules.
- Develop a Python application for project registration, investigator assignment, grant and budget entry, milestone tracking, expenditure entry, publication linkage and reporting.
- Generate six reports through SQL views and validate the system with positive and negative test cases.

1.3 Scope and Assumptions

Table 1 defines what the project covers and what it does not, together with the assumptions made.

Table 1. Scope and Assumptions

Item	Description
In Scope	Records of departments, researchers, projects, investigators, funding agencies, grants, milestones, expenditure, publications, patents and reports; business rules enforced inside the database; a Python application for data entry, search, update, controlled deletion and reports.
Out of Scope / Exclusions	Payroll or full accounting integration; online grant application and approval workflow; cloud deployment and mobile application; plagiarism or peer-review checking.
Assumptions	Each grant belongs to one project and one funding agency; the budget of a project is the sum of its grants; all amounts are in Indian rupees; a publication is identified by its DOI; investigators are existing researchers of the institution.

1.4 Proposed Solution Strategy

The proposed solution replaces the separate files with one MySQL database and a Python application. Table 2 summarizes the users, modules, workflow, rules and outputs of the solution.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Research Office Administrator, Principal Investigator, Finance Officer and Department Head (read-only view).
Major Application Modules	Project registration, investigator assignment, grant and budget entry, milestone tracking, expenditure entry, publication linkage, search and reports.
Proposed Workflow	Register project, assign investigators, enter grant, track milestones, record expenditure, link publications and run reports.
Important Business Rules	Valid project dates; expenditure within budget; milestone dates within the project period; unique publication identifiers; positive grant values.
Expected Reports / Outputs	Project progress, budget utilization, milestones, researcher output, funding-agency portfolio and publications.

1.5 Functional Requirements

The functional requirements of the system are listed in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Register, view and update a research project with title, department, start and end dates and status.	High
FR-02	Assign investigators to a project with the role PI, Co-PI or Member.	High
FR-03	Record grants (agency, amount, sanction date) against a project; the amount must be positive.	High
FR-04	Record milestones with due dates inside the project period and update their status.	High
FR-05	Record expenditure against a grant; total spending must not exceed the grant amount.	High

Req. ID	Functional Requirement	Remarks / Priority
FR-06	Record publications with a unique DOI and link them to projects and authors.	High
FR-07	Record patents and reports linked to projects.	Medium
FR-08	Search projects, researchers, grants, milestones, expenditure and publications by key fields.	Medium
FR-09	Generate six reports from database views.	High
FR-10	Validate input, show clear error messages, roll back failed transactions and block deletion of referenced records.	High

1.6 Entity-Relationship Diagram (ERD)

The ER model [1] of the system is shown in Figure 1. It has 11 principal entities: department, researcher, investigator, project, funding agency, grant, milestone, expenditure, publication, patent and report, with project as the central entity. Each box is a table of the final relational schema with its primary key (PK), foreign keys (FK) and the unique DOI column; the cardinality of each relationship is marked 1 or N. The many-to-many relationships between researcher and project, researcher and publication, and project and publication are resolved by project_investigator, publication_author and project_publication. Every foreign key is NOT NULL, so participation is total on the many side of each one-to-many relationship. The full attributes are given in Table 4 and Table 8.

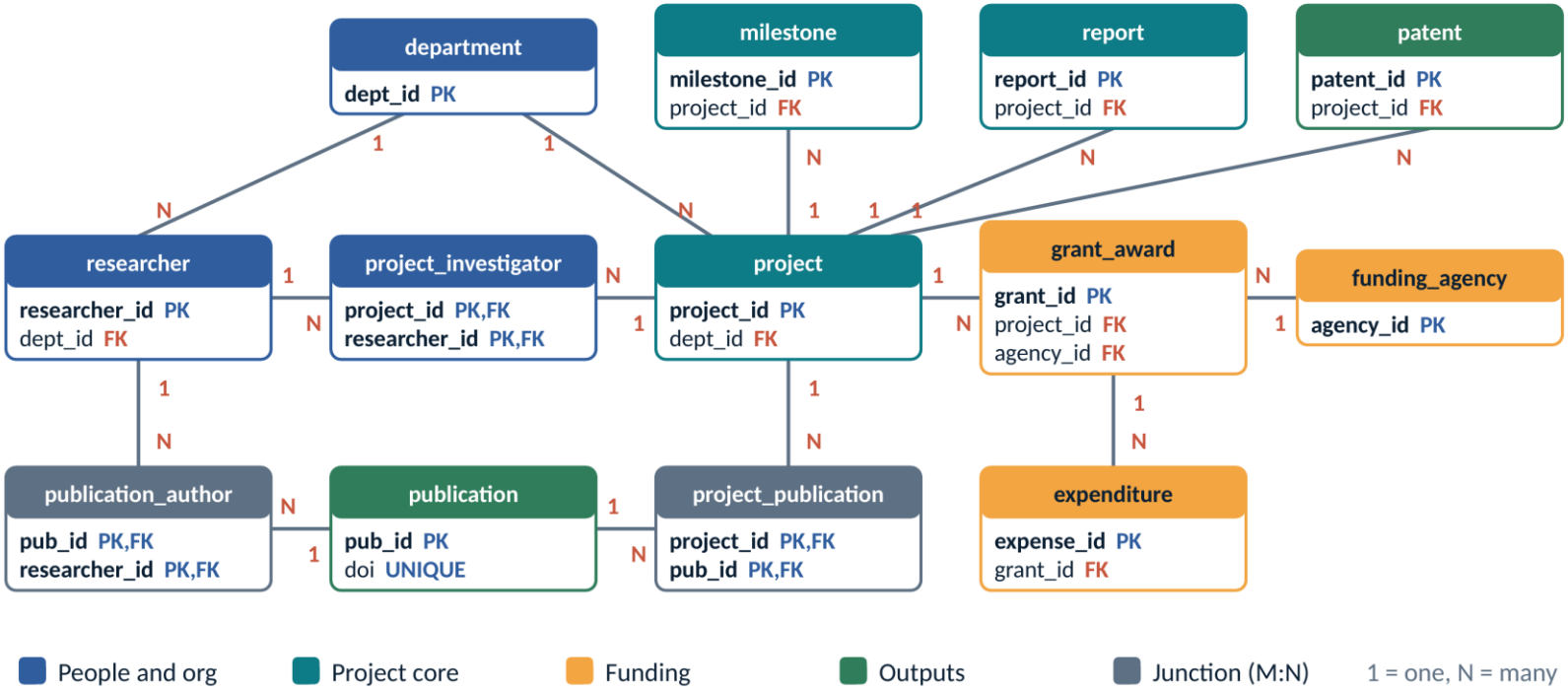


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 summarizes each entity and relationship with its major attributes, cardinality and the rule it carries.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Department	dept_id, dept_name, hod_name, building	Academic unit that hosts projects and employs researchers.
Researcher	researcher_id, full_name, email, designation, dept_id	Person who can be an investigator or a publication author.
Investigator	project_id, researcher_id, role, assigned_on	Assignment of a researcher to a project as PI, Co-PI or Member.
Project	project_id, title, dept_id, start_date, end_date, status	Central entity. Rule: end date after start date.
Funding Agency	agency_id, agency_name, agency_type, contact_email	Organization that sanctions grants.
Grant	grant_id, project_id, agency_id, grant_amount, sanction_date	Sanctioned funding. Rule: amount greater than zero.
Milestone	milestone_id, project_id, title, due_date, status	Planned deliverable. Rule: due date within project dates.
Expenditure	expense_id, grant_id, category, amount, spent_on	Spending charged to a grant. Rule: total within grant amount.
Publication	pub_id, title, venue, pub_year, doi	Paper produced by research. Rule: DOI is unique.
Patent	patent_id, project_id, title, patent_no, filing_date, status	Patent resulting from a project. Rule: patent number is unique.
Report	report_id, project_id, report_type, generated_on	Report generated for a project.
Employs / Hosts	Department-Researcher 1:N; Department-Project 1:N	Every researcher and every project belongs to one department.
Assigned to	Researcher-Project M:N (via project_investigator)	A researcher can work on many projects and a project has many investigators.
Funds / Charged to	Agency-Grant 1:N; Project-Grant 1:N; Grant-Expenditure 1:N	Money flows from agency to grant to expenditure.
Tracks / Yields	Project-Milestone 1:N; Project-Report 1:N; Project-Patent 1:N	A project owns its milestones, reports and patents.
Linked to / Authored	Project-Publication M:N; Researcher-Publication M:N	Resolved by project_publication and publication_author.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model was converted into relations using the standard mapping and the relational model of data [2]. Each entity became a table with its identifier as primary key; each one-to-many relationship became a foreign key on the many side; and each many-to-many relationship became a junction table whose primary key combines the two foreign keys. This gives the 13 tables of Table 5.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	department	Academic departments.
2	researcher	Researchers and their department.
3	funding_agency	Agencies that sanction grants.
4	project	Research projects with dates and status.
5	project_investigator	Junction: researchers assigned to projects, with role.
6	grant_award	Grants sanctioned to projects by agencies (named grant_award because GRANT is a reserved SQL word).
7	milestone	Project milestones with due dates and status.
8	expenditure	Spending recorded against grants.
9	publication	Publications identified by a unique DOI.
10	project_publication	Junction: publications linked to projects.
11	publication_author	Junction: authors of publications, with author order.
12	patent	Patents filed from projects.
13	report	Reports generated for projects.

2.2 Keys and Integrity Constraints

Table 6 lists the keys and integrity constraints of each table and the business rule each one enforces. All foreign keys use the default referential action (RESTRICT), so a parent row cannot be deleted while child rows refer to it. CHECK constraints cannot read other tables in MySQL, so the two cross-table rules are enforced by triggers.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
department	Primary Key	dept_id	Each department is identified uniquely.
	Unique, Not Null	dept_name	No two departments share a name.
researcher	Primary Key	researcher_id	Each researcher is identified uniquely.
	Foreign Key	dept_id → department	Every researcher belongs to a real department.
	Unique, Not Null	email	One record per official email address.
funding_agency	Primary Key	agency_id	Each agency is identified uniquely.
	Unique, Not Null	agency_name, agency_type	No duplicate agencies.
project	Primary Key	project_id	Each project is identified uniquely.
	Foreign Key	dept_id → department	A project is hosted by a real department.
	Check	end_date > start_date	Valid project dates.
	Default	status = 'Planned'	New projects start as planned.
project_investigator	Primary Key	(project_id, researcher_id)	A researcher is assigned once to a project.
	Foreign Key	project_id, researcher_id	Only real projects and researchers can be linked.
	Default	role = 'Member'	Default role of an investigator.
grant_award	Primary Key	grant_id	Each grant is identified uniquely.
	Foreign Key	project_id, agency_id	A grant belongs to a real project and agency.
	Check	grant_amount > 0	Positive grant values.
milestone	Primary Key	milestone_id	Each milestone is identified uniquely.
	Foreign Key	project_id → project	A milestone belongs to a real project.
	Trigger	due_date	Milestone dates within project dates.
	Default	status = 'Pending'	New milestones start as pending.
expenditure	Primary Key	expense_id	Each expense is identified uniquely.
	Foreign Key	grant_id → grant_award	Every expense is charged to a real grant.
	Check	amount > 0	Positive expense amounts.
	Trigger	amount	Expenditure within budget.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
publication	Primary Key	pub_id	Each publication is identified uniquely.
	Unique, Not Null	doi	Unique publication identifiers.
project_publication	Primary Key	(project_id, pub_id)	A publication is linked once to a project.
	Foreign Key	project_id, pub_id	Only real projects and publications can be linked.
publication_author	Primary Key	(pub_id, researcher_id)	An author is listed once per publication.
	Foreign Key	pub_id, researcher_id	Only real publications and researchers.
	Default	author_order = 1	Default position in the author list.
patent	Primary Key	patent_id	Each patent is identified uniquely.
	Foreign Key	project_id → project	A patent belongs to a real project.
	Unique, Not Null	patent_no	No duplicate patent numbers.
report	Primary Key	report_id	Each report is identified uniquely.
	Foreign Key	project_id → project	A report belongs to a real project.
	Default	generated_on = current date	Report date is set automatically.

2.3 Functional Dependencies

The important functional dependencies used to justify the design are listed below; the determinant on the left of each arrow is a key of its table.

- department: dept_id → dept_name, hod_name, building
- researcher: researcher_id → full_name, email, designation, dept_id
- funding_agency: agency_id → agency_name, agency_type, contact_email
- project: project_id → title, dept_id, start_date, end_date, status
- project_investigator: (project_id, researcher_id) → role, assigned_on
- grant_award: grant_id → project_id, agency_id, grant_amount, sanction_date
- milestone: milestone_id → project_id, title, due_date, status
- expenditure: expense_id → grant_id, category, amount, spent_on
- publication: pub_id → title, venue, pub_year, doi, and doi → pub_id (candidate key)
- publication_author: (pub_id, researcher_id) → author_order
- patent: patent_id → project_id, title, patent_no, filing_date, status
- report: report_id → project_id, report_type, generated_on

2.4 Normalization up to Third Normal Form (3NF)

The design started from a single flat sheet of the kind a research office keeps today, with one row for every combination of project, researcher, grant and publication. Such a sheet is not in 1NF because it holds repeating groups (several investigators, grants and DOIs for one project), and it suffers from update, insert and delete anomalies [3], [4]. In the 1NF step, values were made atomic and each repeating group moved into its own table. In the 2NF step, attributes that depended on only part of a composite key were separated: for example, a researcher's name and email depend only on researcher_id and stay in researcher, while role depends on both project_id and researcher_id and lives in project_investigator. In the 3NF step, attributes that depended on a non-key attribute were moved out: department and agency details are stored once and referenced by key, and the project budget is not stored at all because it is the sum of the project's grants. Table 7 shows the final status. Every determinant in every relation is a candidate key, so all 13 relations are also in BCNF.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
department	Yes	Yes	Yes	Yes	Department details stored once.
researcher	Yes	Yes	Yes	Yes	Department name moved out to department (no transitive dependency).
funding_agency	Yes	Yes	Yes	Yes	Agency details stored once and referenced by key.
project	Yes	Yes	Yes	Yes	Budget not stored; derived from grants. Department details referenced by key.
project_investigator	Yes	Yes	Yes	Yes	Role depends on both parts of the key (no partial dependency).
grant_award	Yes	Yes	Yes	Yes	Agency name and type not repeated; separated from project.
milestone	Yes	Yes	Yes	Yes	Repeating group of milestones moved out of the project sheet.
expenditure	Yes	Yes	Yes	Yes	Repeating group of expenses moved out; charged to a grant.
publication	Yes	Yes	Yes	Yes	DOI is a second candidate key; no non-key dependencies.
project_publication	Yes	Yes	Yes	Yes	Junction table holding only its key.
publication_author	Yes	Yes	Yes	Yes	Author order depends on the whole key.
patent	Yes	Yes	Yes	Yes	Patent details separated from the project.
report	Yes	Yes	Yes	Yes	Report details separated from the project.

2.5 Data Dictionary

Table 8 is the data dictionary of all 13 tables. Table names are shown on the first row of each group.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
department	dept_id	INT	PK, AUTO_INCREMENT	Unique department identifier
	dept_name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the department
	hod_name	VARCHAR(100)	-	Head of the department
	building	VARCHAR(100)	-	Building of the department
researcher	researcher_id	INT	PK, AUTO_INCREMENT	Unique researcher identifier
	full_name	VARCHAR(100)	NOT NULL	Full name of the researcher
	email	VARCHAR(120)	NOT NULL, UNIQUE	Official email address
	designation	VARCHAR(60)	-	Academic designation
	dept_id	INT	FK, NOT NULL	Department of the researcher
funding_agency	agency_id	INT	PK, AUTO_INCREMENT	Unique agency identifier
	agency_name	VARCHAR(120)	NOT NULL, UNIQUE	Name of the agency
	agency_type	VARCHAR(50)	NOT NULL	Type, for example government or industry
	contact_email	VARCHAR(120)	-	Contact email of the agency
project	project_id	INT	PK, AUTO_INCREMENT	Unique project identifier
	title	VARCHAR(200)	NOT NULL	Project title
	dept_id	INT	FK, NOT NULL	Hosting department
	start_date	DATE	NOT NULL	Project start date
	end_date	DATE	NOT NULL, CHECK	Project end date, after start date
	status	ENUM	NOT NULL, DEFAULT 'Planned'	Planned, Active or Completed
project_investigator	project_id	INT	PK, FK	Project the researcher works on
	researcher_id	INT	PK, FK	Assigned researcher
	role	ENUM	NOT NULL, DEFAULT 'Member'	PI, Co-PI or Member
	assigned_on	DATE	NOT NULL, DEFAULT	Date of assignment (current date)
grant_award	grant_id	INT	PK, AUTO_INCREMENT	Unique grant identifier
	project_id	INT	FK, NOT NULL	Project that receives the grant
	agency_id	INT	FK, NOT NULL	Agency that sanctions the grant

Table	Attribute	Data Type	Key / Constraint	Description
	grant_amount	DECIMAL(12,2)	NOT NULL, CHECK	Sanctioned amount, greater than 0
	sanction_date	DATE	NOT NULL	Date of sanction
milestone	milestone_id	INT	PK, AUTO_INCREMENT	Unique milestone identifier
	project_id	INT	FK, NOT NULL	Project of the milestone
	title	VARCHAR(150)	NOT NULL	Milestone description
	due_date	DATE	NOT NULL	Due date, within project dates (trigger)
	status	ENUM	NOT NULL, DEFAULT 'Pending'	Pending, In Progress or Completed
expenditure	expense_id	INT	PK, AUTO_INCREMENT	Unique expense identifier
	grant_id	INT	FK, NOT NULL	Grant charged
	category	VARCHAR(50)	NOT NULL	Expense category
	amount	DECIMAL(12,2)	NOT NULL, CHECK	Amount spent, greater than 0
	spent_on	DATE	NOT NULL	Date of expense
publication	pub_id	INT	PK, AUTO_INCREMENT	Unique publication identifier
	title	VARCHAR(250)	NOT NULL	Title of the publication
	venue	VARCHAR(150)	-	Journal or conference
	pub_year	YEAR	NOT NULL	Year of publication
	doi	VARCHAR(100)	NOT NULL, UNIQUE	Digital Object Identifier
project_publication	project_id	INT	PK, FK	Linked project
	pub_id	INT	PK, FK	Linked publication
publication_author	pub_id	INT	PK, FK	Publication
	researcher_id	INT	PK, FK	Author
	author_order	TINYINT	NOT NULL, DEFAULT 1	Position in the author list
patent	patent_id	INT	PK, AUTO_INCREMENT	Unique patent identifier
	project_id	INT	FK, NOT NULL	Project of the patent
	title	VARCHAR(200)	NOT NULL	Patent title
	patent_no	VARCHAR(50)	NOT NULL, UNIQUE	Patent or application number
	filing_date	DATE	NOT NULL	Date of filing

Table	Attribute	Data Type	Key / Constraint	Description
	status	ENUM	NOT NULL, DEFAULT 'Filed'	Filed, Published or Granted
report	report_id	INT	PK, AUTO_INCREMENT	Unique report identifier
	project_id	INT	FK, NOT NULL	Project of the report
	report_type	VARCHAR(60)	NOT NULL	Kind of report
	generated_on	DATE	NOT NULL, DEFAULT	Date generated (current date)

2.6 Implementation of Logical Design: DDL and Sample Data

The database was implemented in MySQL, as summarized in Table 9. The representative CREATE TABLE statements below show how the keys and the CHECK, UNIQUE, NOT NULL and DEFAULT constraints are declared; the cross-table rules are enforced by a trigger. The complete script is in Appendix A.

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL
DBMS Version	8.0 (CHECK constraints need 8.0.16 or later)
Number of Tables	13 tables, 6 views, 4 triggers
Sample Data	Realistic records for all tables: 286 rows (for example 5 departments, 15 researchers, 12 projects, 14 grants, 60 expenditures, 25 publications)

Representative DDL

```
CREATE TABLE project (
  project_id INT AUTO_INCREMENT PRIMARY KEY,
  title      VARCHAR(200) NOT NULL,
  dept_id    INT NOT NULL,
  start_date DATE NOT NULL,
  end_date   DATE NOT NULL,
  status ENUM('Planned','Active','Completed')
         NOT NULL DEFAULT 'Planned',
  CONSTRAINT chk_dates CHECK (end_date > start_date),
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

CREATE TABLE grant_award (
  grant_id INT AUTO_INCREMENT PRIMARY KEY,
  project_id INT NOT NULL,
  agency_id  INT NOT NULL,
  grant_amount DECIMAL(12,2) NOT NULL CHECK (grant_amount > 0),
  sanction_date DATE NOT NULL,
  FOREIGN KEY (project_id) REFERENCES project(project_id),
  FOREIGN KEY (agency_id) REFERENCES funding_agency(agency_id)
);
```

```
CREATE TABLE publication (
  pub_id   INT AUTO_INCREMENT PRIMARY KEY,
  title    VARCHAR(250) NOT NULL,
  venue    VARCHAR(150),
  pub_year YEAR NOT NULL,
  doi      VARCHAR(100) NOT NULL,
  CONSTRAINT uq_pub_doi UNIQUE (doi)
);
```

Trigger for the budget rule

```
CREATE TRIGGER trg_exp_budget
BEFORE INSERT ON expenditure FOR EACH ROW
BEGIN
  IF (SELECT IFNULL(SUM(amount), 0) FROM expenditure
       WHERE grant_id = NEW.grant_id) + NEW.amount >
     (SELECT grant_amount FROM grant_award
       WHERE grant_id = NEW.grant_id) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Expenditure exceeds grant';
  END IF;
END
```

Representative sample data

```
SELECT project_id AS id, title, start_date, end_date, status
FROM project LIMIT 4;

+----+---------------------------+------------+------------+--------+
| id | title                     | start_date | end_date   | status |
+----+---------------------------+------------+------------+--------+
| 1  | AI Crop Disease Detection | 2026-01-01 | 2027-12-31 | Active |
| 2  | Low-Cost Water Purifier   | 2024-07-01 | 2026-06-30 | Active |
| 3  | Wearable ECG Monitor      | 2025-03-01 | 2027-02-28 | Active |
| 4  | Biodegradable Packaging   | 2024-01-15 | 2026-12-31 | Active |
+----+---------------------------+------------+------------+--------+

SELECT grant_id AS id, project_id AS project, agency_id AS agency,
       grant_amount AS amount, sanction_date AS sanctioned
FROM grant_award LIMIT 4;

+----+---------+--------+------------+------------+
| id | project | agency | amount     | sanctioned |
+----+---------+--------+------------+------------+
| 1  | 1       | 2      | 1800000.00 | 2026-01-15 |
| 2  | 2       | 5      | 1250000.00 | 2024-08-10 |
| 3  | 3       | 1      | 2200000.00 | 2025-04-01 |
| 4  | 4       | 3      | 1500000.00 | 2024-02-20 |
+----+---------+--------+------------+------------+
```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application is written in Python and is used by the research office, investigators, finance staff and department heads. It does not use static data: every screen reads from and writes to the MySQL database through a database connector, so all the rules defined in Chapter 2 apply to the data that users enter. The application supports registration of projects, assignment of investigators, entry of grants, milestones, expenditure and publications, search, update, controlled deletion and six reports. When a rule is broken, the database raises an error and the application turns it into a clear message and leaves the data unchanged.

3.2 Development Environment and Database Connectivity

Table 10 lists the development environment. The application connects to MySQL using MySQL Connector/Python [6]. Every statement that uses user input is a parameterised query, so values are passed separately from the SQL text; multi-step saves run inside a transaction that is committed on success and rolled back on any error. The listing below shows the pattern for recording an expense.

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3
DBMS	MySQL 8.0
IDE / Framework / GUI Toolkit	IDE and GUI toolkit used
Database Connector / Driver	MySQL Connector/Python [6]
Operating Environment	Operating system
Repository Link	https://github.com/VarunKumarGorre/dbms_project

Representative connection and error-handling code

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user=USER,
                               password=PASSWORD, database="research_db")
cur = conn.cursor()
try:
    cur.execute("INSERT INTO expenditure (grant_id, category, amount, "
                "spent_on) VALUES (%s, %s, %s, %s)",
                (grant_id, category, amount, spent_on))
    conn.commit()
except mysql.connector.Error as err:
    conn.rollback()
    show_error(err.msg)
```

3.3 Application Modules and Database Mapping

Table 11 maps each application module to the tables it uses and the main database operations it performs.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Project Registration	Create and maintain projects	project, department	Insert, select, update
Investigator Assignment	Assign PI, Co-PI and members	project_investigator, researcher, project	Insert, select, delete
Grant and Budget Entry	Record grants against projects	grant_award, funding_agency, project	Insert, select, update
Milestone Tracking	Plan milestones and update status	milestone, project	Insert, select, update
Expenditure Entry	Record spending against a grant	expenditure, grant_award	Insert (transaction), select
Publication Linkage	Record publications; link to projects and authors	publication, project_publication, publication_author	Insert, select, link
Search	Find records by key fields	project, researcher, grant_award, milestone, expenditure, publication	Select with filters
Reports	Show the six report views	v_project_progress, v_budget_utilization and four other views	Select

3.4 CRUD Operations

Table 12 shows how the four CRUD operations are provided and where evidence appears in the report.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Project registration, investigator assignment, grant, milestone, expenditure and publication entry	project, project_investigator, grant_award, milestone, expenditure, publication	Figure 2, Figure 3
Read / Search	Search and list screens; report screens	All main tables and the six views	Figure 4, Figure 5
Update	Edit a project, change milestone status, correct a grant	project, milestone, grant_award	Figure 4
Delete / Controlled Removal	Delete a record after confirmation; blocked by foreign keys when dependent records exist	Any table without dependent rows	Table 15 (TC-07)

3.5 Business Rules, Validation and Error Handling

The application prevents invalid input in two layers. It first validates required fields and number and date formats and shows a message at once. Any rule that is still broken is stopped by the database constraints or triggers, and the application reports the error text to the user. Table 13 lists the rules, how each is implemented and the message shown.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Valid project dates	Application checks end date against start date; CHECK constraint chk_dates in project	End date must be after start date
Positive grant values	Application accepts only numbers above zero; CHECK constraint on grant_amount	Grant amount must be positive
Expenditure within budget	Trigger trg_exp_budget raises SQLSTATE 45000; the transaction is rolled back	Expenditure exceeds grant (Figure 3)
Milestone dates within project	Trigger trg_ms_dates compares the due date with the project dates	Milestone outside project
Unique publication identifier	UNIQUE constraint uq_pub_doi; duplicate-key error is caught by the application	DOI already exists
Input validation and SQL injection	Required fields and number and date formats are checked; all SQL uses parameterised queries	Field-level error messages
Controlled deletion and transactions	User confirms deletion; foreign keys block removal of referenced rows; multi-step saves use commit and rollback	Record is in use and cannot be deleted

3.6 Reports and Database Operations

The six reports are created as database views, so the application only selects from them. Table 14 lists the SQL concept behind each report and the output it gives.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Project progress	JOIN, GROUP BY, SUM and COUNT (v_project_progress)	Milestones done, total and percentage for each project
Budget utilization	Derived tables, LEFT JOIN, SUM, ROUND (v_budget_utilization)	Sanctioned amount, spent amount and percentage used (Figure 5)
Milestones	JOIN with CASE to flag overdue items (v_milestone_status)	Milestone list with an Overdue flag
Researcher output	Nested subquery and HAVING (v_researcher_output)	Researchers with above-average publications
Funding-agency portfolio	LEFT JOIN, SUM, ORDER BY (v_agency_portfolio)	Number of grants and total amount per agency
Publications	M:N join through the junction table (v_project_publications)	Papers linked to each project, with DOI

3.7 Application Screens

Figures 2 to 5 show the main screens. Figure 2 shows data entry for a new project and its investigators. Figure 3 shows expenditure entry, a transaction, and the rejection of an expense above the grant. Figure 4 shows milestone tracking, search and update together with publication linkage. Figure 5 shows the Budget Utilization report.

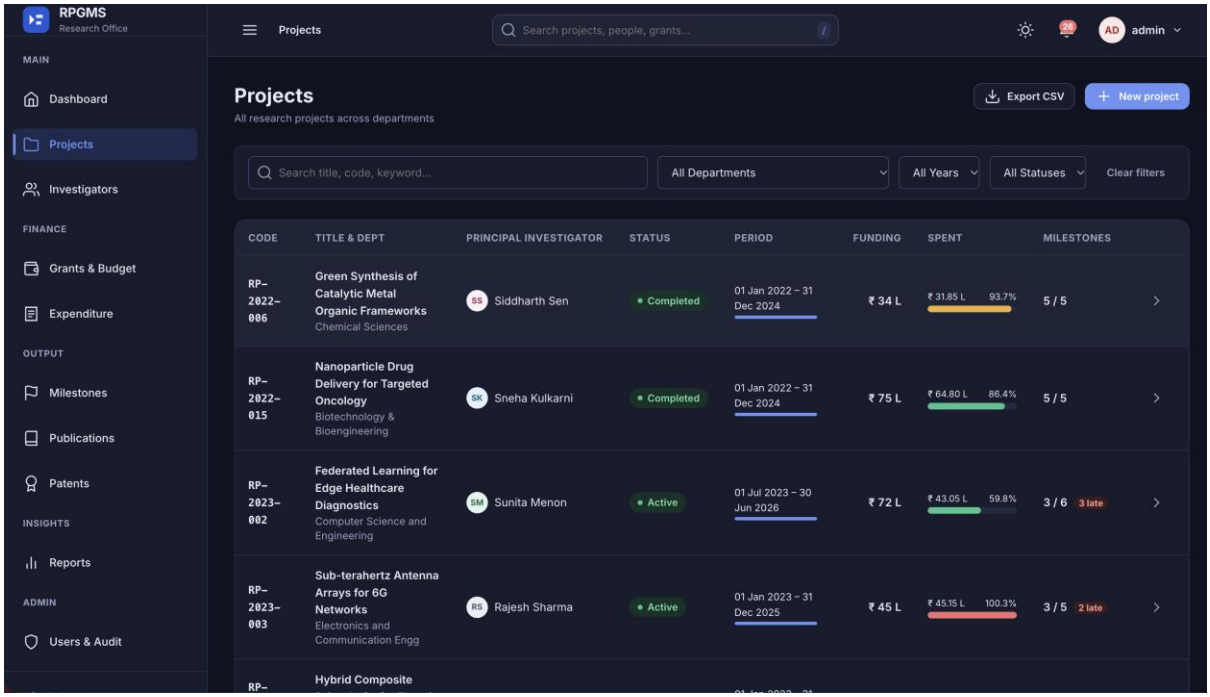


Figure 2. Project Registration and Investigator Assignment Screen

Figure 3. Expenditure Entry and Budget-Rule Rejection

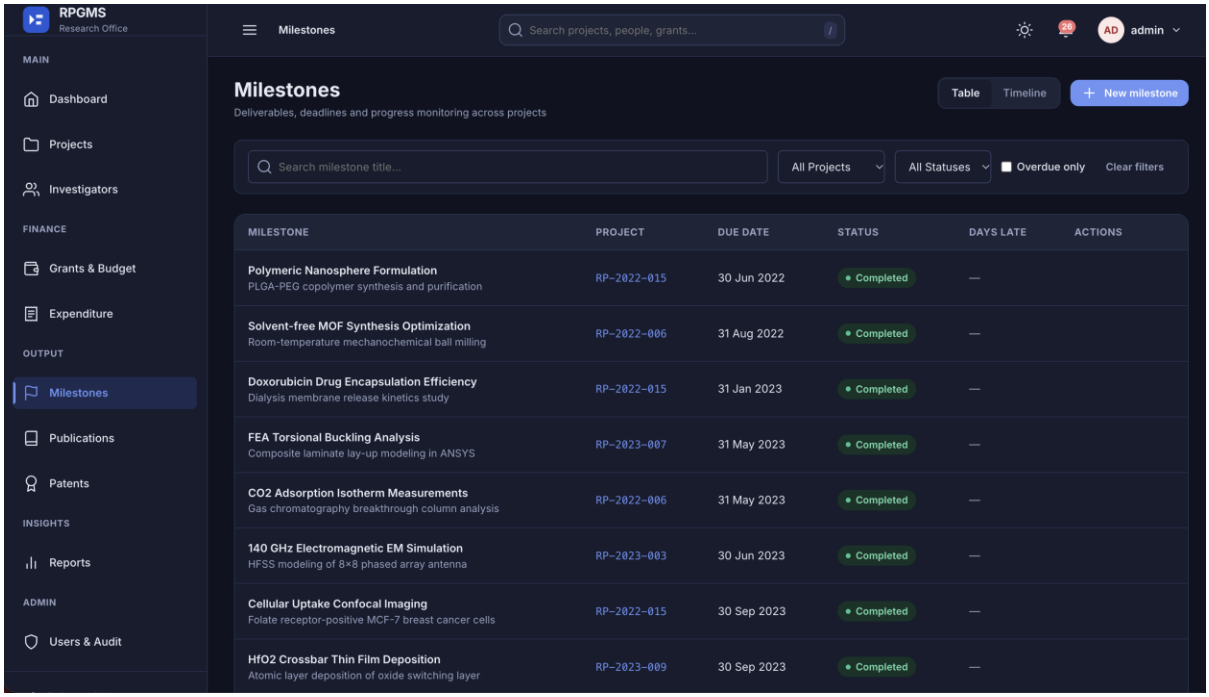


Figure 4. Milestone Tracking and Publication Linkage Screen

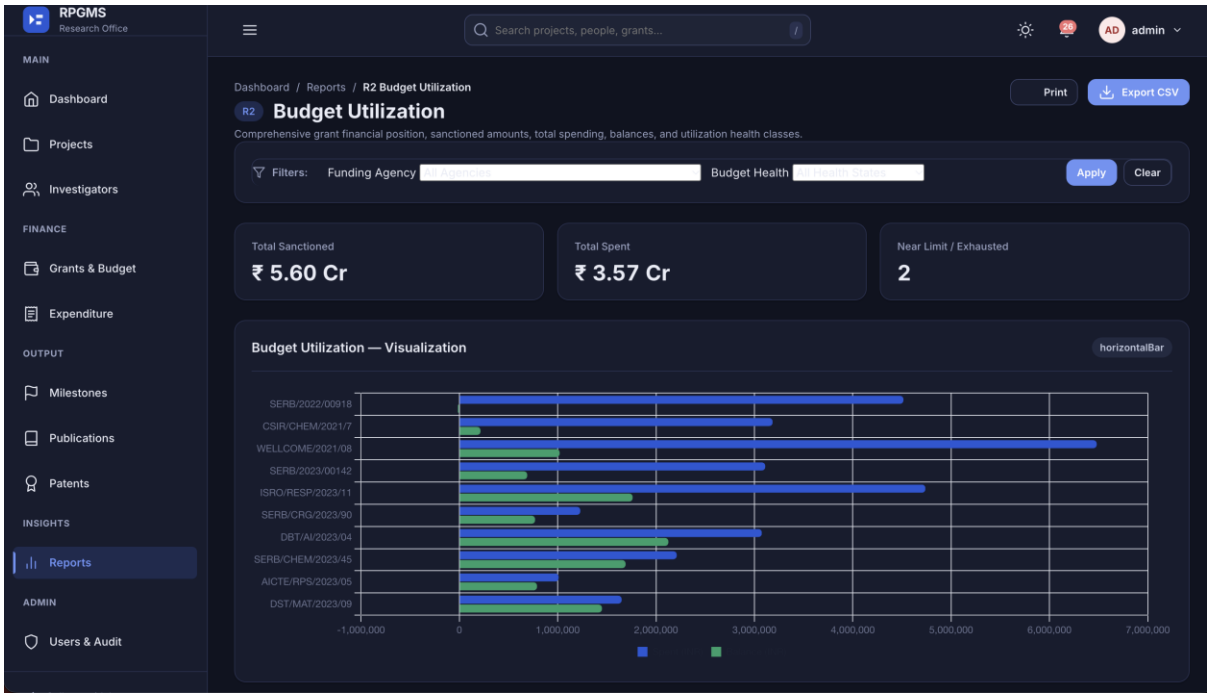


Figure 5. Budget Utilization Report

3.8 Testing and Results

Table 15 lists the positive and negative test cases used to check the application against the database design.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Project registration	Enter a valid title, department and dates	Project is saved and listed	Project successfully registered and displayed in the project list with the entered details.	Pass
TC-02	Project dates	End date earlier than start date	Rejected: End date must be after start date	System displayed a validation message: “End date must be after start date.” The invalid project was not saved.	Pass
TC-03	Grant value	Grant amount 0	Rejected: Grant amount must be positive	System displayed a validation message: “Grant amount must be greater than zero.” The invalid grant record was not saved.	Pass
TC-04	Expenditure budget	Expense larger than the remaining grant	Rejected by trigger: Expenditure exceeds grant	System rejected the expenditure exceeding the remaining grant balance. The transaction was rolled back, and the expenditure record was not saved.	Pass
TC-05	Milestone dates	Due date after the project end date	Rejected: Milestone outside project	System rejected the milestone with a due date beyond the project end date and displayed a validation message.	Pass
TC-06	Unique DOI	Publication with an existing DOI	Rejected: DOI already exists	System rejected the duplicate DOI and displayed a message indicating that the DOI already exists. No duplicate publication was saved.	Pass
TC-07	Controlled deletion	Delete a project that has grants	Blocked by foreign key	System prevented the deletion of a project linked to existing grant records. The project and its related records remained unchanged.	Pass

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-08	Search and report	Search a project by keyword; open Budget Utilization	Matching rows listed; percentages shown	System displayed projects matching the search keyword and generated the Budget Utilization report with the calculated expenditure percentages.	Pass

All eight test cases behaved as designed: valid data was accepted and every business-rule violation was rejected, either by the application or by the database, with a clear message and no partial data left behind. The tests show that the application uses the designed database rather than static data and that the five business rules are enforced by the database itself. The reports reflect the stored data.

CHAPTER 4

CONCLUSION

This project addressed the lack of an integrated record system in research administration. A relational database for research projects, publications and grants was designed from an ER model with eleven principal entities, converted into thirteen tables, normalized to Third Normal Form and implemented in MySQL with keys and CHECK, UNIQUE, NOT NULL and DEFAULT constraints. Two triggers enforce the rules that span more than one table: expenditure within budget and milestone dates within the project.

A Python application built on this design supports project registration, investigator assignment, grant and budget entry, milestone tracking, expenditure recording, publication linkage, search and six reports. Testing with valid and invalid inputs showed that rule violations are rejected and reported clearly, and that reports are computed from the stored data. The main learning was that enforcing integrity in the database, rather than only in the application, keeps the data reliable however it is entered, and that careful normalization makes the reports straightforward to write.

4.1 Limitations and Future Enhancements

- The application has no login or role-based access control; all users see the same screens.
- Amounts are in a single currency and the budget is held only at grant level, without separate budget heads such as equipment or travel.
- Reports are viewed on screen; charts and export to PDF or Excel are not included.
- The system is built for one institution and one database server and has not been tested with very large data volumes or many simultaneous users.

REFERENCES

[1] P. P.-S. Chen, "The entity-relationship model—toward a unified view of data," ACM Trans. Database Syst., vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.

[2] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] Oracle Corporation, MySQL 8.0 Reference Manual. Accessed: Oct. 15, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] Oracle Corporation, MySQL Connector/Python Developer Guide. Accessed: Oct. 15, 2026. [Online]. Available: https://dev.mysql.com/doc/connector-python/en/

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete script is submitted as the file research_grant_db.sql. It creates the 13 tables with their constraints, the indexes, the triggers and the six views. Its contents are reproduced below.

```
-- research_grant_db.sql : complete database script (MySQL 8.0)
CREATE DATABASE research_db;
USE research_db;

CREATE TABLE department (
  dept_id   INT AUTO_INCREMENT PRIMARY KEY,
  dept_name VARCHAR(100) NOT NULL UNIQUE,
  hod_name  VARCHAR(100),
  building  VARCHAR(100)
);
CREATE TABLE researcher (
  researcher_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name   VARCHAR(100) NOT NULL,
  email       VARCHAR(120) NOT NULL UNIQUE,
  designation VARCHAR(60),
  dept_id     INT NOT NULL,
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
CREATE TABLE funding_agency (
  agency_id     INT AUTO_INCREMENT PRIMARY KEY,
  agency_name   VARCHAR(120) NOT NULL UNIQUE,
  agency_type   VARCHAR(50) NOT NULL,
  contact_email VARCHAR(120)
);
CREATE TABLE project (
  project_id INT AUTO_INCREMENT PRIMARY KEY,
  title      VARCHAR(200) NOT NULL,
  dept_id    INT NOT NULL,
  start_date DATE NOT NULL,
  end_date   DATE NOT NULL,
  status ENUM('Planned','Active','Completed')
         NOT NULL DEFAULT 'Planned',
  CONSTRAINT chk_dates CHECK (end_date > start_date),
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
CREATE TABLE project_investigator (
  project_id    INT NOT NULL,
  researcher_id INT NOT NULL,
  role ENUM('PI','Co-PI','Member') NOT NULL DEFAULT 'Member',
  assigned_on DATE NOT NULL DEFAULT (CURRENT_DATE),
  PRIMARY KEY (project_id, researcher_id),
  FOREIGN KEY (project_id) REFERENCES project(project_id),
  FOREIGN KEY (researcher_id) REFERENCES researcher(researcher_id)
);
CREATE TABLE grant_award (
  grant_id INT AUTO_INCREMENT PRIMARY KEY,
  project_id INT NOT NULL,
  agency_id  INT NOT NULL,
  grant_amount DECIMAL(12,2) NOT NULL CHECK (grant_amount > 0),
  sanction_date DATE NOT NULL,
  FOREIGN KEY (project_id) REFERENCES project(project_id),
  FOREIGN KEY (agency_id) REFERENCES funding_agency(agency_id)
);
CREATE TABLE milestone (
  milestone_id INT AUTO_INCREMENT PRIMARY KEY,
  project_id   INT NOT NULL,
  title        VARCHAR(150) NOT NULL,
  due_date     DATE NOT NULL,
  status ENUM('Pending','In Progress','Completed')
         NOT NULL DEFAULT 'Pending',
  FOREIGN KEY (project_id) REFERENCES project(project_id)
);
CREATE TABLE expenditure (
  expense_id INT AUTO_INCREMENT PRIMARY KEY,
  grant_id   INT NOT NULL,
  category   VARCHAR(50) NOT NULL,
  amount     DECIMAL(12,2) NOT NULL CHECK (amount > 0),
  spent_on   DATE NOT NULL,
```

```
  FOREIGN KEY (grant_id) REFERENCES grant_award(grant_id)
);
CREATE TABLE publication (
  pub_id   INT AUTO_INCREMENT PRIMARY KEY,
  title    VARCHAR(250) NOT NULL,
  venue    VARCHAR(150),
  pub_year YEAR NOT NULL,
  doi      VARCHAR(100) NOT NULL,
  CONSTRAINT uq_pub_doi UNIQUE (doi)
);
CREATE TABLE project_publication (
  project_id INT NOT NULL,
  pub_id     INT NOT NULL,
  PRIMARY KEY (project_id, pub_id),
  FOREIGN KEY (project_id) REFERENCES project(project_id),
  FOREIGN KEY (pub_id) REFERENCES publication(pub_id)
);
CREATE TABLE publication_author (
  pub_id        INT NOT NULL,
  researcher_id INT NOT NULL,
  author_order  TINYINT NOT NULL DEFAULT 1,
  PRIMARY KEY (pub_id, researcher_id),
  FOREIGN KEY (pub_id) REFERENCES publication(pub_id),
  FOREIGN KEY (researcher_id) REFERENCES researcher(researcher_id)
);
CREATE TABLE patent (
  patent_id   INT AUTO_INCREMENT PRIMARY KEY,
  project_id  INT NOT NULL,
  title       VARCHAR(200) NOT NULL,
  patent_no   VARCHAR(50) NOT NULL UNIQUE,
  filing_date DATE NOT NULL,
  status ENUM('Filed','Published','Granted')
         NOT NULL DEFAULT 'Filed',
  FOREIGN KEY (project_id) REFERENCES project(project_id)
);
CREATE TABLE report (
  report_id    INT AUTO_INCREMENT PRIMARY KEY,
  project_id   INT NOT NULL,
  report_type  VARCHAR(60) NOT NULL,
  generated_on DATE NOT NULL DEFAULT (CURRENT_DATE),
  FOREIGN KEY (project_id) REFERENCES project(project_id)
);

-- Indexes on frequently searched columns
CREATE INDEX idx_project_status ON project(status);
CREATE INDEX idx_milestone_due  ON milestone(due_date);
CREATE INDEX idx_pub_year       ON publication(pub_year);
CREATE INDEX idx_researcher_nm  ON researcher(full_name);
CREATE INDEX idx_expense_date   ON expenditure(spent_on);

-- Triggers for cross-table rules
DELIMITER //
CREATE TRIGGER trg_exp_budget
BEFORE INSERT ON expenditure FOR EACH ROW
BEGIN
  IF (SELECT IFNULL(SUM(amount), 0) FROM expenditure
       WHERE grant_id = NEW.grant_id) + NEW.amount >
     (SELECT grant_amount FROM grant_award
       WHERE grant_id = NEW.grant_id) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Expenditure exceeds grant';
  END IF;
END//
CREATE TRIGGER trg_exp_budget_upd
BEFORE UPDATE ON expenditure FOR EACH ROW
BEGIN
  IF (SELECT IFNULL(SUM(amount), 0) FROM expenditure
       WHERE grant_id = NEW.grant_id
```

```
        AND expense_id <> NEW.expense_id) + NEW.amount >
     (SELECT grant_amount FROM grant_award
       WHERE grant_id = NEW.grant_id) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Expenditure exceeds grant';
  END IF;
END//
CREATE TRIGGER trg_ms_dates
BEFORE INSERT ON milestone FOR EACH ROW
BEGIN
  IF NEW.due_date NOT BETWEEN
     (SELECT start_date FROM project WHERE project_id = NEW.project_id)
     AND
     (SELECT end_date FROM project WHERE project_id = NEW.project_id)
  THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Milestone outside project';
  END IF;
END//
CREATE TRIGGER trg_ms_dates_upd
BEFORE UPDATE ON milestone FOR EACH ROW
BEGIN
  IF NEW.due_date NOT BETWEEN
     (SELECT start_date FROM project WHERE project_id = NEW.project_id)
     AND
     (SELECT end_date FROM project WHERE project_id = NEW.project_id)
  THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Milestone outside project';
  END IF;
END//
DELIMITER ;

-- Report views
CREATE VIEW v_project_progress AS
SELECT p.title,
       SUM(m.status = 'Completed') AS done,
       COUNT(*) AS total,
       ROUND(100 * SUM(m.status = 'Completed') / COUNT(*)) AS pct
FROM project p JOIN milestone m USING (project_id)
GROUP BY p.project_id, p.title;

CREATE VIEW v_budget_utilization AS
SELECT p.title, g.sanctioned,
       IFNULL(e.spent, 0) AS spent,
       ROUND(100 * IFNULL(e.spent, 0) / g.sanctioned) AS pct
FROM project p
JOIN (SELECT project_id, SUM(grant_amount) AS sanctioned
      FROM grant_award GROUP BY project_id) g USING (project_id)
LEFT JOIN (SELECT ga.project_id, SUM(x.amount) AS spent
           FROM expenditure x JOIN grant_award ga USING (grant_id)
           GROUP BY ga.project_id) e USING (project_id);

CREATE VIEW v_milestone_status AS
SELECT p.title AS project, m.title AS milestone, m.due_date,
       CASE WHEN m.status <> 'Completed' AND m.due_date < CURDATE()
            THEN 'Overdue' ELSE m.status END AS state
FROM milestone m JOIN project p USING (project_id);

CREATE VIEW v_researcher_output AS
SELECT r.full_name, COUNT(*) AS pubs
FROM researcher r JOIN publication_author pa USING (researcher_id)
GROUP BY r.researcher_id, r.full_name
HAVING COUNT(*) >
  (SELECT AVG(n) FROM (SELECT COUNT(*) AS n
                       FROM publication_author
                       GROUP BY researcher_id) t);
```

```
CREATE VIEW v_agency_portfolio AS
SELECT f.agency_name, COUNT(g.grant_id) AS grants,
       SUM(g.grant_amount) AS total
FROM funding_agency f LEFT JOIN grant_award g USING (agency_id)
GROUP BY f.agency_id, f.agency_name
ORDER BY total DESC;

CREATE VIEW v_project_publications AS
SELECT p.title AS project, b.title AS publication,
       b.venue, b.pub_year, b.doi
FROM project p
JOIN project_publication pp USING (project_id)
JOIN publication b USING (pub_id);
```

APPENDIX B - APPLICATION SOURCE CODE

The complete Python source code is submitted as the project folder and is available at https://github.com/VarunKumarGorre/dbms_project. It is not reproduced in this report.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

The statements below are the negative test inputs behind test cases TC-02 to TC-07 in Table 15. Each statement must be rejected by the database.

```
-- Negative test inputs used for TC-02 to TC-07 (each must fail)
-- TC-02: end date before start date          -> CHECK chk_dates
INSERT INTO project (title, dept_id, start_date, end_date)
VALUES ('Test', 1, '2026-06-01', '2026-01-01');

-- TC-03: zero grant amount                   -> CHECK on grant_amount
INSERT INTO grant_award (project_id, agency_id, grant_amount, sanction_date)
VALUES (1, 2, 0, CURDATE());

-- TC-04: expense above the grant             -> trigger trg_exp_budget
INSERT INTO expenditure (grant_id, category, amount, spent_on)
VALUES (1, 'Equipment', 99999999.00, CURDATE());

-- TC-05: milestone after project end         -> trigger trg_ms_dates
INSERT INTO milestone (project_id, title, due_date)
VALUES (1, 'Late demo', '2035-01-01');

-- TC-06: duplicate DOI                       -> UNIQUE uq_pub_doi
INSERT INTO publication (title, venue, pub_year, doi)
VALUES ('Copy', 'Journal', 2026, '<an existing DOI>');

-- TC-07: delete a project that has grants    -> foreign key (error 1451)
DELETE FROM project WHERE project_id = 1;
```